Forge

An autonomous, MCP-powered coding agent for the terminal

Project report

Toby Wong

Generative AI · Independent Study (5010)

October 2026

1. System overview

Forge is an autonomous command-line coding agent. The user types a task. Forge sends it, with the schemas of every available tool, to an LLM, executes the tool calls the model makes, feeds the results back, and repeats until the model gives a final answer (or a 25-step budget runs out). Every tool call is shown in the terminal. In `confirm` mode, any tool that changes something shows a preview first (a real diff for edits) and asks y / n / always.

Components: CLI (prompt_toolkit + Rich) → agentic loop → provider abstraction (LangChain) and tool registry → MCP client manager → three MCP servers (filesystem, Context7, my RAG server) plus one native `run_command` tool. All diagrams are in section 2.

Requirements coverage

Requirement	Where it lives
Agentic loop (reason → act → observe → repeat)	`forge/agent.py` : streaming LLM call, tool dispatch, step budget, malformed-call recovery
Tool calling: read/edit/write files, shell, search, retrieval	filesystem MCP tools, native `run_command` , Context7 and RAG tools (`forge/tools.py`)
Confirm vs auto-execute	`/mode confirm|auto` , `--auto` ; confirm mode previews diffs, file contents and commands
Visible tool calls	every call shows server › tool + arguments, then the result, its timing and a preview
Provider abstraction: Ollama + cloud/hosted	`forge/providers.py` : Ollama (gemma4:e2b) and an OpenAI-compatible llama.cpp server (Qwen3.8-27B), switchable with `/model`
MCP client with multiple servers, dynamic tools	`forge/mcp_client.py` + `mcp_servers.json` ; tools discovered with `list_tools` at startup
Filesystem server	`@modelcontextprotocol/server-filesystem` , sandboxed to the workspace
External resource server	Context7 (`@upstash/context7-mcp`)
Custom RAG MCP server, one-time ingestion	`rag_server/` : `ingest.py` builds a persistent Chroma store once; `server.py` serves it
Advanced RAG technique	Fusion retrieval (NirDiamant/RAG_Techniques), evaluated in `rag_server/evaluate.py`
Streaming CLI with status indicators	live Markdown streaming, thinking indicator, tool spinners, toolbar with model and mode
State and sequence diagrams	section 2.3 and 2.4 (sources in `docs/planning/`)
LLM comparison	`scripts/compare_models.py` → `docs/evaluation/model_comparison.md` (section 4)
Optional extension: view and confirm changes before edits	`edit_file` dry run shows the diff before the user approves

2. Architecture

2.1 Original architecture (planning phase)

Source: planning/02_architecture.md, committed before implementation.

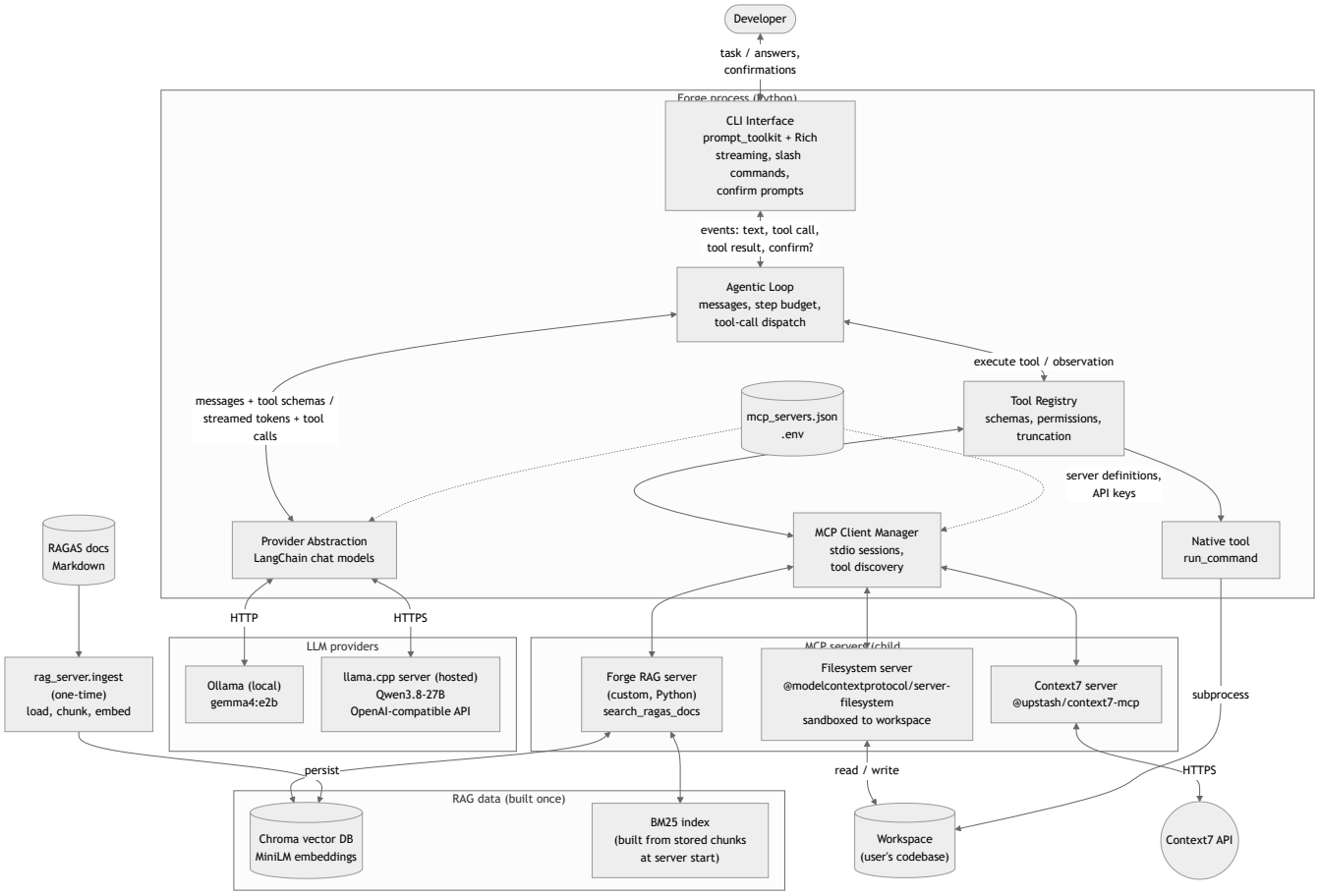


Figure 1. 2.1 Original architecture (planning phase)

Component responsibilities

Component	Responsibility
CLI Interface	Reads tasks and slash commands; streams the model's text; renders each tool call (server, tool, arguments) and result; asks for confirmation in `confirm` mode
Agentic Loop	Holds the conversation; calls the LLM with the tool schemas; executes the returned tool calls in order; appends observations; repeats until the model answers without tool calls or the step budget runs out
Provider Abstraction	Builds a LangChain chat model for `ollama` or `llamacpp` from the config; `bind_tools` gives every provider the same tool interface
Tool Registry	Merges MCP tools and native tools; converts MCP JSON Schemas to function schemas; classifies tools as read-only or mutating; truncates outputs

Component	Responsibility
MCP Client Manager	Starts each configured server, runs `initialize` + `list_tools`, routes `call_tool` to the owning session, and shuts the servers down cleanly
Filesystem server	Reads, writes, edits (with diff/dry-run) and searches files, restricted to the workspace
Context7 server	Up-to-date documentation for public libraries
Forge RAG server	Fusion retrieval (BM25 + vectors) over the RAGAS documentation

2.2 Architecture as built, and what changed

Source: architecture_as_built.md.

The original planning-phase diagram is in planning/02_architecture.md. The core design held up. The changes below came from problems found while testing; new or changed parts are highlighted in the diagram.

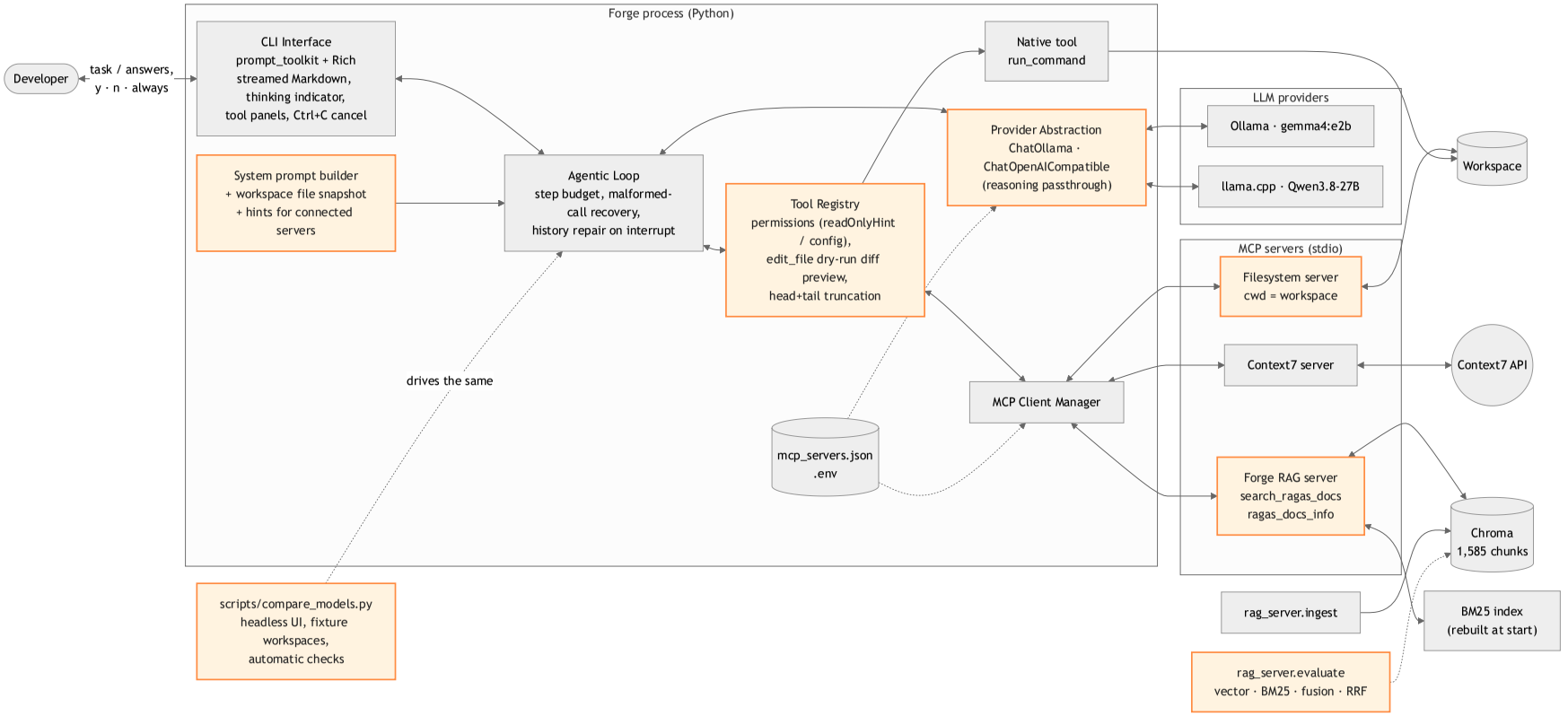


Figure 2. 2.2 Architecture as built, and what changed

What changed and why

Change	Why
Workspace snapshot in the system prompt (new `prompts.py` builder)	In the first end-to-end test, Gemma invented `src/calculator.py` (copying an example path in my own prompt) instead of editing `calculator.py`. With a short file tree of the real workspace in the prompt, the same task went from 10 steps with a wrong file to 5 correct steps.
Server-specific prompt hints only for connected servers	A precaution: if a server fails to start (or is disabled), the prompt never tells the model to call tools that aren't loaded.
`ChatOpenAICompatible` **subclass**	LangChain's `ChatOpenAI` drops llama.cpp's non-standard `reasoning_content` field. The subclass keeps it, so the CLI can show a thinking indicator for both providers.
Permission source: MCP `readOnlyHint` **annotations**	The plan used a hand-made list. The filesystem server already annotates each tool, so Forge uses that and falls back to a per-server `read_only` flag in the config (Context7, RAG).
`edit_file` **dry run decides whether to ask**	If the dry run fails (the text isn't found), the real edit would fail too, so the error goes straight back to the model instead of asking the user to approve a doomed change.
Filesystem server started with `cwd = workspace`	Models often use relative paths; these now resolve inside the workspace.
Head+tail truncation of tool output	Test failures and tracebacks are at the end of command output; keeping only the head hid them.
Second RAG tool `ragas_docs_info`	Lets the agent (and the demo) show what is indexed: commit, page and chunk counts, embedding model.
RRF retriever and an evaluation module	Added to measure fusion retrieval against vector-only, BM25-only and rank-based fusion (see `evaluation/rag_retrieval.md`).
Benchmark harness drives the same `Agent`	Because the loop reports through a UI protocol, a headless UI can run it for the model comparison with no CLI code.

Nothing was removed from the plan. The state and sequence diagrams in `planning/` still describe the runtime behaviour.

2.3 State diagram

Source: planning/03_state_diagram.md.

This shows how information moves through Forge, from startup through one task (an "agent turn") and back to idle.

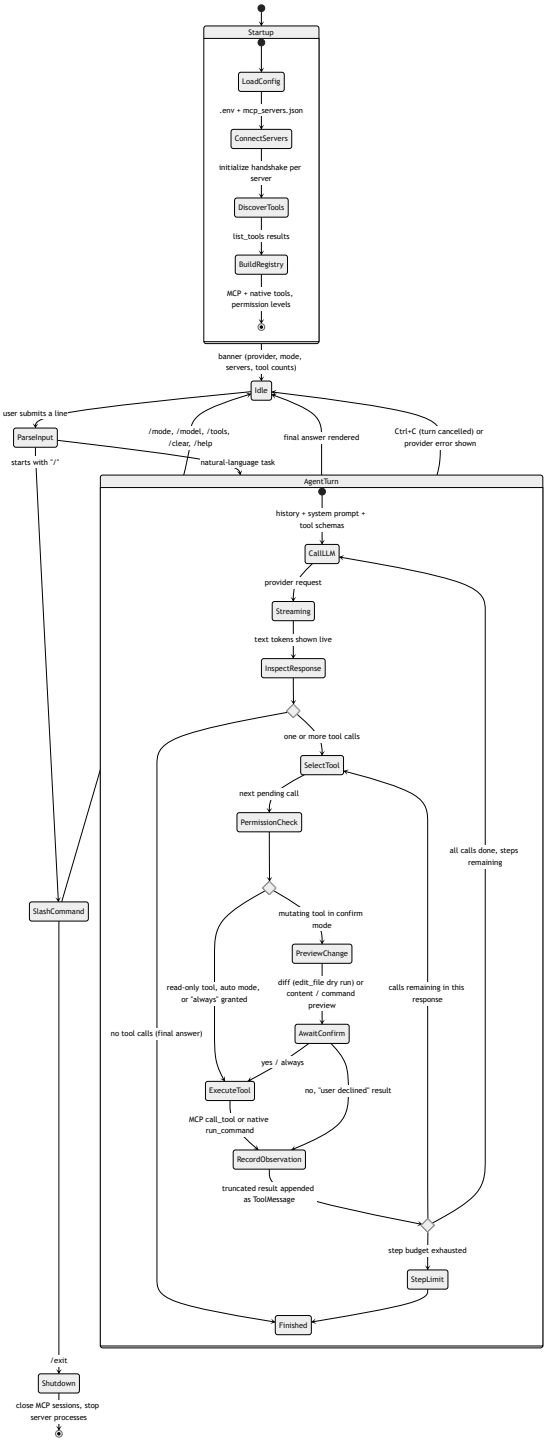


Figure 3. 2.3 State diagram

Notes

- **Observations always go back to the model**, including errors and declined confirmations. The model sees what happened and can choose a different approach (for example, `create_directory` after a failed `write_file`).
- The **step budget** (default 25 LLM calls per task) stops runaway loops with small models.
- `/clear` empties the conversation history but keeps MCP connections; `/model` swaps the provider without restarting the servers.

2.4 Sequence diagrams

Source: planning/04_sequence_diagrams.md.

Four end-to-end operations:

1. **Startup and dynamic tool discovery**
2. **Fixing a bug in confirm mode**: filesystem server, diff preview, confirmation prompts, native shell tool
3. **Writing code grounded in the RAG server** (auto mode): custom RAG server, fusion retrieval
4. **Looking up external docs, and a declined change**: Context7 server, user says "no", model adapts

Participants: **User**, **CLI Interface**, **Agentic Loop**, **Tools / MCP Client** (tool registry + MCP client manager), the **MCP servers**, and the **LLM** (behind the provider abstraction).

---

1. Startup and dynamic tool discovery

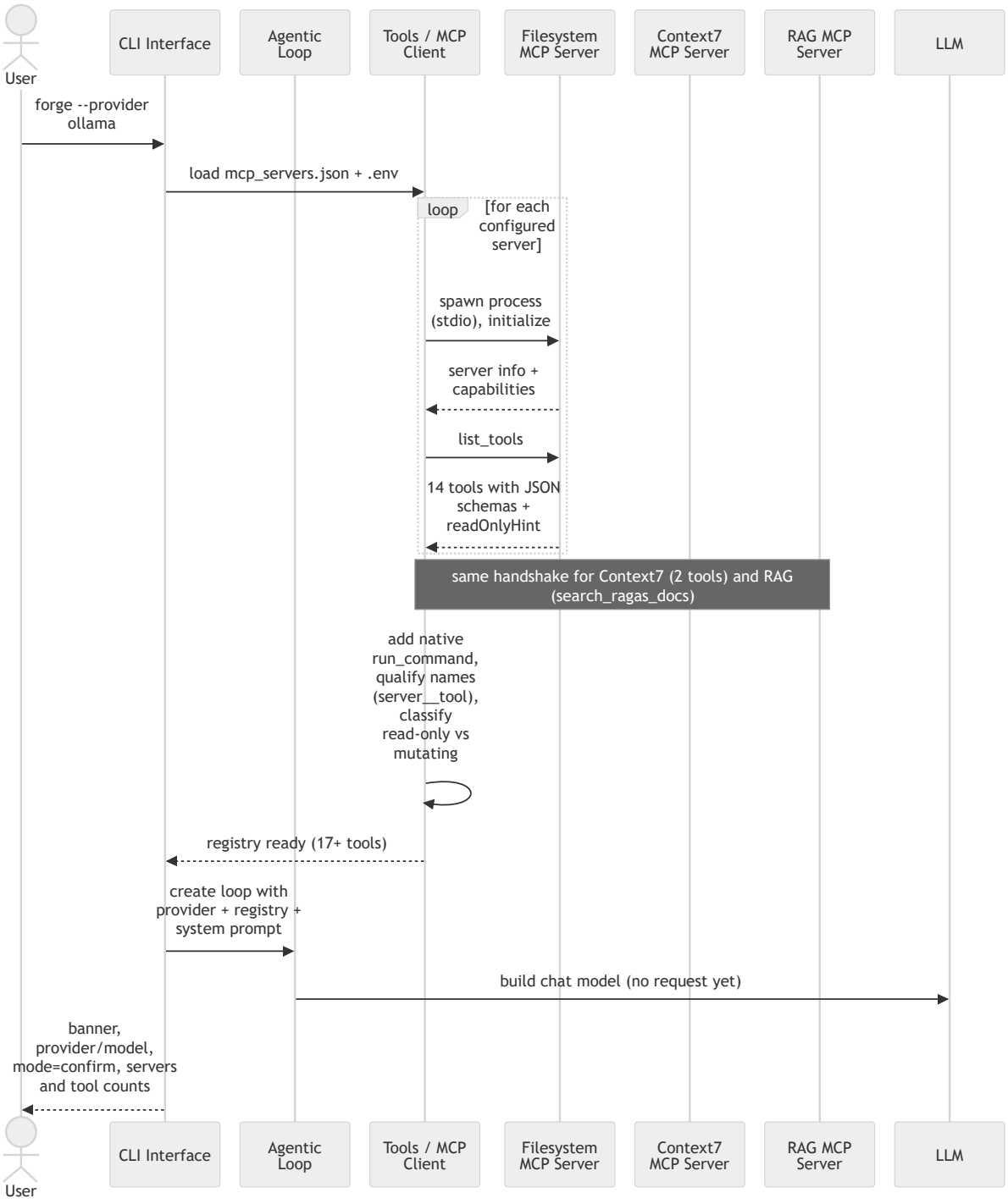


Figure 4. 1. Startup and dynamic tool discovery

2. Fixing a bug in confirm mode

Task: "*divide() in calculator.py crashes on zero. Fix it to raise a ValueError and run the tests.*"

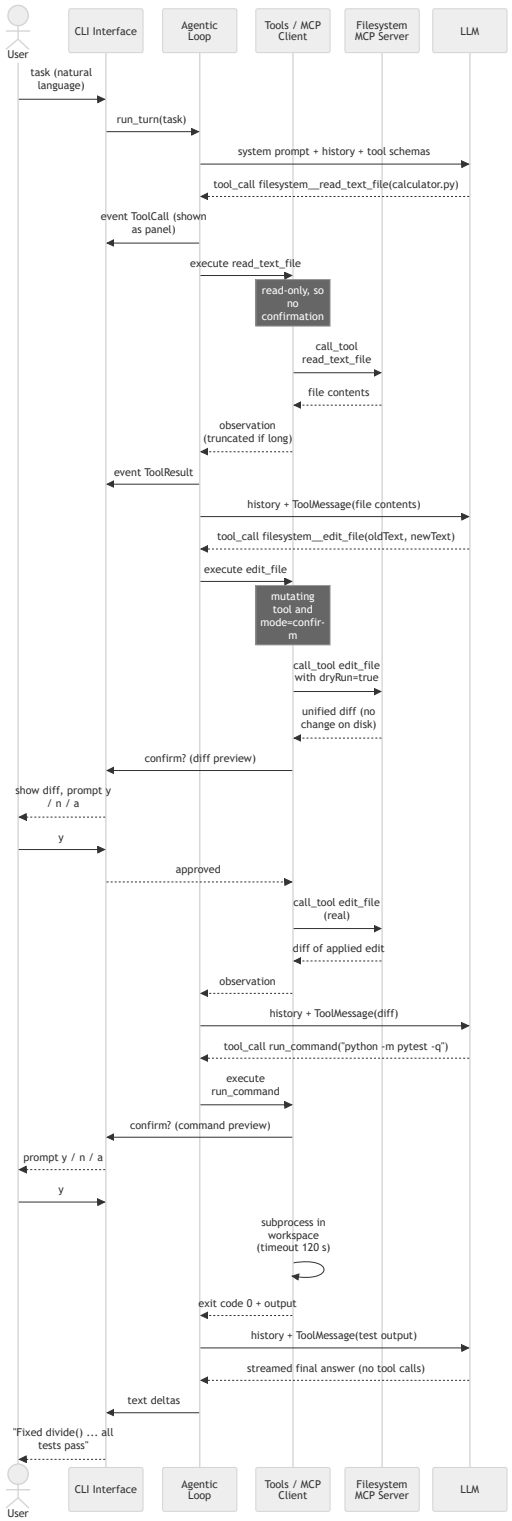


Figure 5. 2. Fixing a bug in confirm mode

3. Writing code grounded in the RAG server (auto mode)

Task: "*Write eval_faithfulness.py that scores one sample with RAGAS' Faithfulness metric.*"

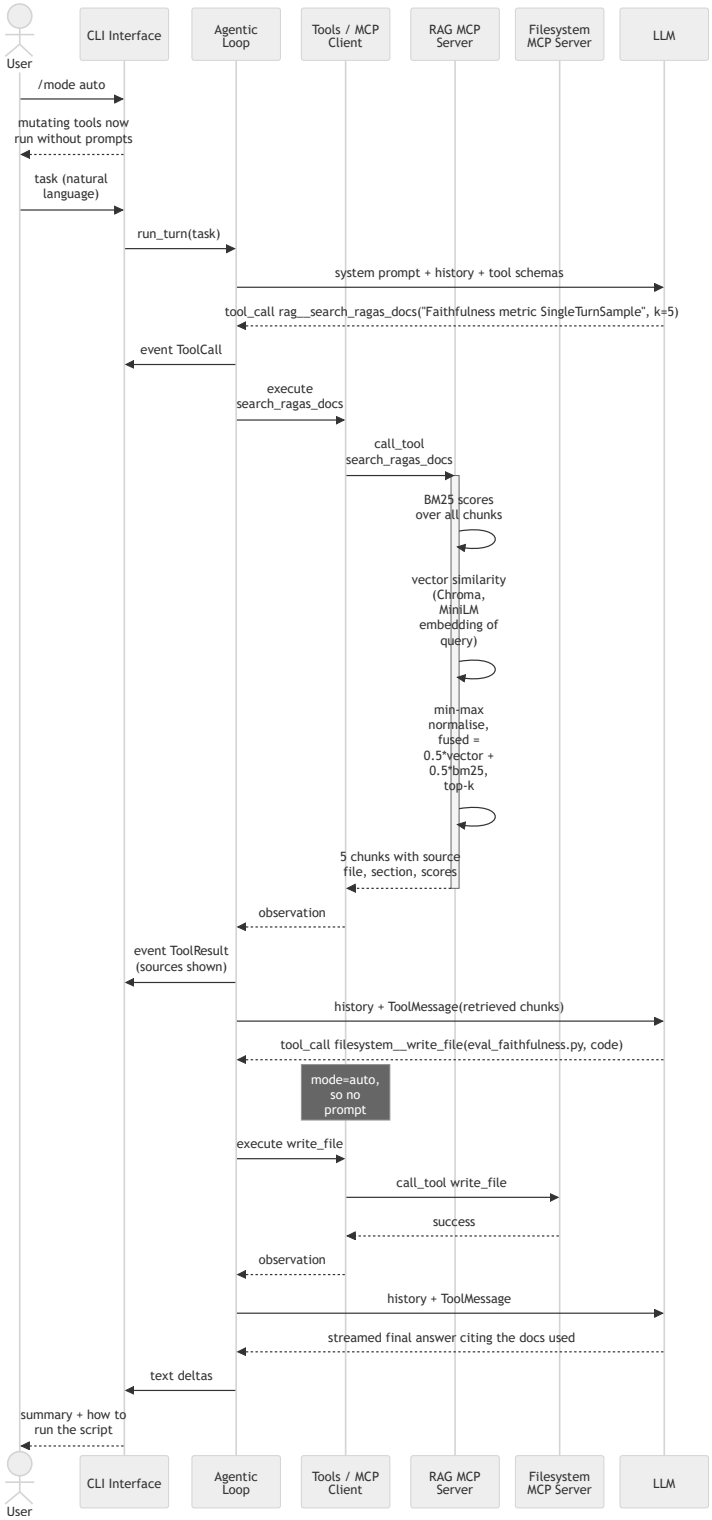


Figure 6. 3. Writing code grounded in the RAG server (auto mode)

4. Looking up external docs, and a declined change

Task: "*Add a LangChain @tool wrapper around slugify() using the current LangChain API.*"

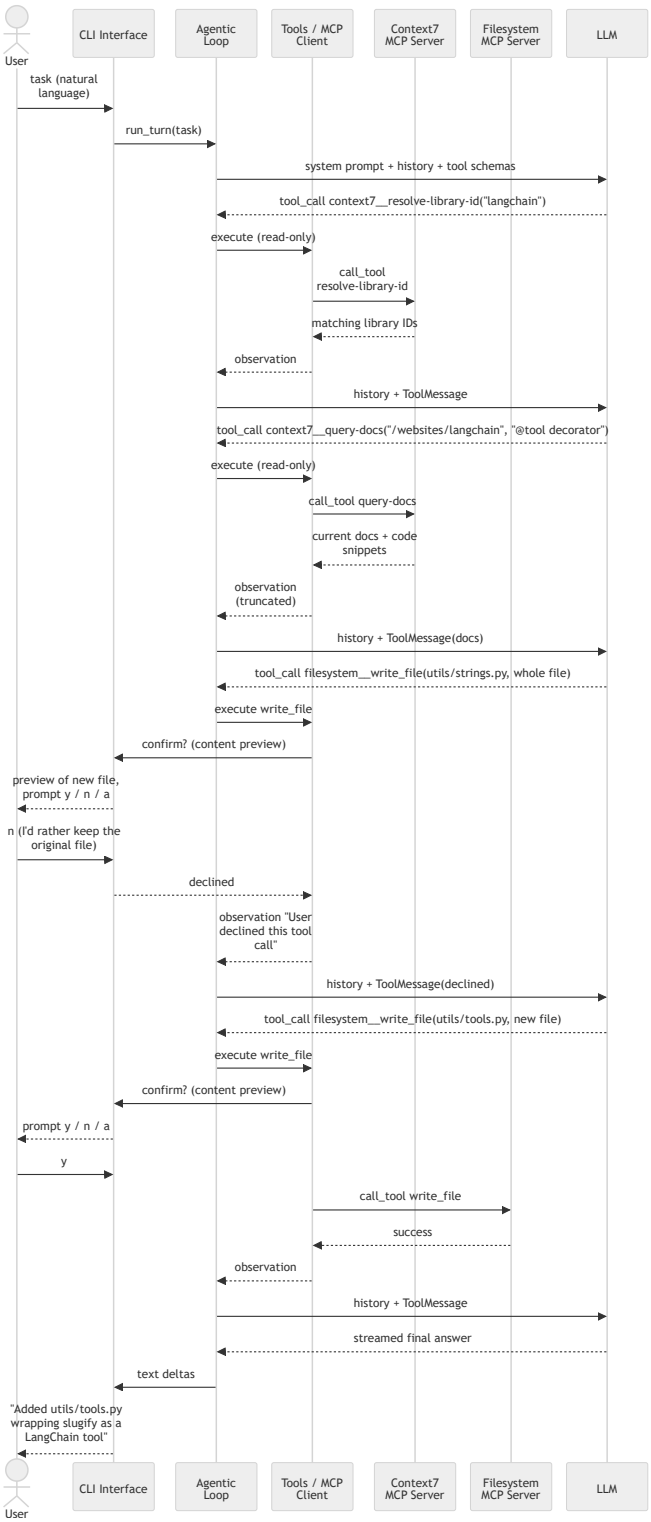


Figure 7. 4. Looking up external docs, and a declined change

3. Design decisions

1. **My own agent loop; LangChain only for models.** LangChain's chat models give one streaming + `bind_tools` interface for Ollama and OpenAI-compatible servers. I wrote the loop myself so confirmation, previews, cancellation and tool visibility sit exactly where I need them.
2. **My own MCP client on the official SDK** (not an adapter library): stdio sessions, `list_tools` discovery, qualified tool names (`server__tool`), JSON Schema → function-calling schema conversion.
3. **Servers are configuration.** `mcp_servers.json` defines all three servers; the RAG server was added without code changes.
4. **Permissions come from MCP annotations.** The filesystem server marks each tool `readOnlyHint` . Docs servers are trusted as read-only in the config. `run_command` always counts as mutating.
5. **RAG: RAGAS docs + fusion retrieval.** 133 pages → 1,585 chunks (Markdown-header split, then 1,200 chars with 150 overlap), MiniLM ONNX embeddings in a persistent Chroma store built once by `python -m rag_server.ingest` . Fusion = 0.5 × normalised vector score + 0.5 × normalised BM25 score. I chose RAGAS because its API changed recently (legacy vs "collections" metrics), so a model's memory of it is often out of date, which is exactly where retrieval should help.
6. **The loop is UI-agnostic.** It reports through callbacks, so the same loop runs the CLI and the headless benchmark.
7. **A workspace snapshot in the system prompt.** A short file tree of the project is included at startup, so the model starts from real paths instead of guessing (see section 4).

4. LLM comparison

Same three tasks, fresh fixture workspace per run, auto mode, all MCP servers connected, scored by **automatic checks** (hidden tests, file inspection, tool traces), not by the model's summary. Full table and per-run traces: evaluation/model_comparison.md.

Task	gemma4:e2b (Ollama, local)	Qwen3.8-27B (llama.cpp)
bugfix: fix divide-by-zero, write and run tests	3/3 success · 5.0 steps · 11 s · 19k tokens in	3/3 success · 4.7 steps · 13 s · 21k tokens in
rag_codegen: RAGAS Faithfulness script from docs	3/3 success · 3.0 steps · 13 s · 13k tokens in	3/3 success · 5.7 steps · 46 s · 46k tokens in
feature: stricter slugify, tests in new tests/ folder	**0/3** (2/6 in two earlier batches) · 11.3 steps · 66 s · 57k tokens in	**3/3** · 6.7 steps · 102 s · 34k tokens in
Total	**6/9**	**9/9**

Failed tool calls per run, averaged over all three tasks: Gemma 2.2, Qwen 0.3. On the feature task alone: Gemma 6.7, Qwen 0.7.

Gemma 4 E2B (local, about 2B parameters):

- It reliably handles short, single-file tasks and uses tools correctly: it reads before editing, creates folders before writing, and runs the tests itself.
- **The typical failure is getting stuck in edit_file retries.** On the feature task, the first edit works, but when the new tests fail it sends `edit_file` calls whose `oldText` no longer matches the file ("Could not find exact match"). It retries, sometimes re-reading the file first, but the replacement text still doesn't match, and it stops with its tests still failing. This is why the benchmark scores runs with hidden tests instead of trusting the model's own report.
- Results vary from run to run on the hard task (0/3 vs 2/6 across batches), so I report several runs.
- The workspace snapshot in the system prompt mattered most. In my first end-to-end test, before the snapshot existed, Gemma invented `src/calculator.py` and built the fix there (10 steps). With the snapshot, the same task took 5 steps on the correct file.

Qwen3.8-27B (hosted on llama.cpp):

- **It recovers from its own mistakes; Gemma doesn't.** Qwen hit the same kinds of errors. In one feature run it wrote `tests/test_strings.py` before creating the folder (`ENOENT` , the same `write_file` quirk). In another, its first test run failed. Both times it fixed the cause (created the folder, edited the code) and reran until the tests passed. Gemma, in the same situation, repeated `edit_file` calls that no longer matched the file.
- **It chose a safer editing strategy.** In all three feature runs Qwen rewrote `utils/strings.py` with `write_file` instead of patching it with exact-match `edit_file` calls. That avoids the failure mode that sank Gemma.

- **It was more thorough.** Its test suites were larger (6–7 tests for the bug fix, 14–17 for the feature, covering empty input, tabs and idempotency), and its final summaries listed the files changed, the command used to verify, and the test results. The automatic checks confirmed that the tests pass in every run.
- **It used the RAG server much more.** It made 3–5 searches per run (Gemma made 1) and called `ragas_docs_info` in two runs. Its code moved toward the current API across runs. Run 1 used the older `LangchainLLMWrapper` + `evaluate()` pattern. Run 2 noted that wrapper is deprecated and switched to `llm_factory`, but kept `evaluate()`. Run 3 noted that the docs label the `SingleTurnSample` + `ragas.metrics.Faithfulness` pattern as legacy and used the current `ragas.metrics.collections` API. My checks don't distinguish legacy from current API, so both models score "success" on that task. A stricter check would have separated them.
- **It interpreted instructions more loosely.** Told "Do not run it" for the RAG script, Qwen still used `run_command` in every run, mostly to syntax-check the file with `python -m py_compile`. It never executed the script, but it is the model that stretched the instruction. In confirm mode the user would have been asked first.
- **Thoroughness costs time.** Qwen was about as fast on the bug fix (13 s vs 11 s) but slower on the RAG task (46 s vs 13 s, because of the extra searches) and the feature task (102 s vs 66 s, mostly from writing long test files; one run produced about 10k output tokens). It needed fewer input tokens on the feature task (34k vs 57k) because it didn't spend steps on failed retries. The timings also depend on the hardware each server runs on, so read them as rough.

Takeaway: on short single-file tasks the small local model is as good and is fast. The 27B model's advantage shows up in multi-step work: recovering from errors, picking robust edit strategies, and making several retrieval queries. What separated the two models was less what they knew than whether they noticed a failure and changed approach.

5. Advanced RAG: fusion retrieval insights

32 hand-labelled questions (15 keyword-style naming exact classes, 17 plain-English paraphrases); a hit is a chunk from a page that answers the question. Full results: evaluation/rag_retrieval.md.

Method	hit@1	hit@5	MRR@10
vector only	0.72	0.88	0.80
BM25 only	0.69	0.84	0.75
fusion (α = 0.5)	0.75	**0.94**	0.82
RRF (rank fusion)	**0.78**	**0.94**	**0.85**

Insights

1. **Fusion beats both parts.** hit@5 went from 0.88 (vector) and 0.84 (BM25) to 0.94. The two methods fail on different questions. Vector search ranked the `ContextEntityRecall` page 8th, while BM25 put it 1st. BM25 missed the summarisation page for a paraphrased question, while vector search put it 1st. Fusion recovers most of these.
2. **On keyword questions fusion is nearly perfect** (hit@5 1.00, MRR 0.96). Class names like `LLMContextRecall` are exactly what BM25 is good at.
3. **Paraphrases are the hard part for all methods** (hit@1 0.59 for vector, BM25 and score fusion). Rank-based fusion (RRF) did better there (0.71), probably because min-max normalisation lets one very confident method dominate the fused score.
4. **The weight matters.** α = 0.75 scored best (MRR 0.90), but it was tuned on the same 32 questions, so I keep the default α = 0.5 and treat 0.75 as an optimistic upper bound. A held-out question set would be needed to choose α properly.
5. **A bug in the reference implementation.** The NirDiamant notebook adds vector scores (in similarity order) to BM25 scores (in corpus order) position by position, so a chunk's vector score can be added to another chunk's BM25 score. My version keys both scores by chunk id before fusing.
6. **Remaining failures are vocabulary gaps.** "Did the retriever find all the information needed?" misses the Context Recall page with every method, because the page talks about "reference claims attributed to the context". Query transformation (e.g. HyDE) would be the next technique to try.
7. **Retrieval quality depends on the agent's query.** In the RAG-grounded coding task, Gemma searched for "...SingleTurnSample". That correctly returned the docs' *legacy* API section, so the generated script used the older API. Retrieval worked, but the model's query steered it. Qwen made 3–5 searches per run. In 2 of 3 runs it explicitly identified deprecated or legacy parts of the API from the retrieved docs and replaced them (fully, with the collections API, in one run). Several targeted queries from the agent made up for what a single query misses, which suggests multi-query retrieval inside the server would help weaker models.

6. Reflection

What worked

- The MCP abstraction: three very different servers (Node filesystem, remote Context7, my Python RAG server) behind one client. Adding the RAG server was a config change.
- Visible tool calls and diff previews: the confirm-mode diff caught a model replacing `divide()` instead of adding a new function.
- Automatic checks in the benchmark, rather than trusting model summaries.
- Building the MCP layer first. Milestone 1 surfaced most of the integration surprises before the agent loop depended on them.

What didn't

- Small local models get stuck in retry loops on exact-match edits. Forge recovers (errors go back to the model), but the model doesn't always learn from them. Gemma solved 0 of 3 feature runs, while Qwen recovered from the same kinds of errors every time.
- My first benchmark check was wrong: it required three separate test methods, which the task never asked for, and failed Gemma for it. I fixed the check and reran everything. Automatic grading needs checking too.
- The RAG task's checks can't tell legacy RAGAS code from current-API code, so they overstate how close the two models were on that task.

Challenges along the way

- **MCP SDK v2 differs from most tutorials:** snake_case attributes (`input_schema` , `is_error`), and FastMCP renamed to `MCPServer` . I worked from the installed SDK's own signatures instead of examples.
- **Async lifecycle:** the SDK's anyio task groups must be closed by the task that opened them, so servers connect one at a time on a shared exit stack, each with its own stack so a failed server is cleaned up on its own.
- **Windows specifics:** `npx` is `npx.cmd` ; server stderr had to go to log files to keep the UI clean; console encodings needed forcing to UTF-8; timed-out shell commands need `taskkill /T` to kill the whole process tree; and Ctrl+C has to cancel only the running task, not Forge.
- **Provider quirks:** LangChain's `ChatOpenAI` drops llama.cpp's `reasoning_content` , so a small subclass keeps it for the thinking indicator.
- **Large tool outputs:** Context7 returns about 4,000 characters per call. Outputs sent back to the model are truncated, keeping the head and tail, because test failures appear at the end.

What I'd improve

- Line-range edits or automatic re-reads after a failed `edit_file` , to break retry loops.
- Query transformation (HyDE) or a re-ranker in the RAG server, plus a held-out set to tune α.
- Session persistence and an `/undo` for file edits (optional extensions in the brief).

- Stricter benchmark checks (e.g. reject deprecated APIs, run the generated RAGAS script against a mock LLM) and more runs per task, since the hard task's results varied between batches.

What I learned from the model comparison

- Grade agents on outcomes, not their own reports. The benchmark checks the files and runs hidden tests itself.
- The harness matters as much as the model. The workspace snapshot, error messages returned as observations, and the `edit_file` dry run made the small model usable at all. The prompt and tool design are part of the agent.
- A hybrid setup makes sense: the local model for quick, single-file edits (free and private), the hosted 27B model for multi-step tasks. Forge's `/model` command switches mid-conversation to support that.

Process

- I completed the project individually.
- Milestone 1 began as an exploratory MCP prototype (filesystem + Context7), presented as a working demo. Doing it first exposed the MCP SDK v2 differences early, and it shaped the plan.
- I then wrote the full plan and the architecture, state and sequence diagrams, and committed them before the milestone code. All later work followed the plan's milestones (M2 agent core → M3 RAG → M4 evaluation), one commit per milestone.
- Working alone, I relied on automatic checks as my second reviewer: the retrieval evaluation and the model benchmark. Every number in this report comes from those scripts, not from reading model output by eye.

Appendix: reproducing the results

```
pip install -r requirements.txt



# one-time: build the RAGAS vector store

python -m rag_server.ingest



# section 5: retrieval evaluation -> docs/evaluation/rag_retrieval.md

python -m rag_server.evaluate



# section 4: model comparison -> docs/evaluation/model_comparison.md

python scripts/compare_models.py -p ollama llamacpp -n 3



# walkthrough of all three MCP servers' tools

python mcp_demo.py --demo



# interactive agent on the sample project

forge -w sandbox
```